



DWN - Unid III

Exercício 1:

Durante o desenvolvimento, uma API ASP.NET Core executa em:

http://localhost:5213

e o frontend React em:

http://localhost:5173

A API registra:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("FrontendDev", policy =>
    {
        policy
            .WithOrigins("http://localhost:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});
```

Após app.Build(), o pipeline executa app.UseCors("FrontendDev") antes dos endpoints. Qual é o papel dessa política quando aplicada?

A)

Autenticar usuários provenientes da porta 5173 e impedir chamadas realizadas por outros tipos de cliente.

B)

Permitir, nas condições definidas, que scripts executados no navegador sob a origem http://localhost:5173 acessem respostas da API, sem substituir autenticação ou autorização.

C)

Fazer com que as portas 5173 e 5213 sejam consideradas a mesma origem pelo navegador.

D)

Criar um proxy interno que encaminha ao ASP.NET Core todas as requisições recebidas pelo React.

E)

Impedir que clientes que não sejam navegadores invoquem os métodos autorizados pela API.

Exercício 2:

O Vite está configurado assim:

```
const backendUrl = 'http://localhost:5213';
```

```
export default defineConfig({  
  server: {  
    port: 5173,  
    proxy: {  
      '/api': {  
        target: backendUrl,  
        changeOrigin: true,  
        secure: false  
      }  
    }  
  }  
});
```

No React:

```
fetch('/api/produtos?page=1');
```

Qual descrição representa corretamente o fluxo durante o desenvolvimento?

A)

fetch identifica automaticamente a porta do ASP.NET Core e ignora o servidor Vite.

B)

A chamada chega primeiro ao ASP.NET Core, que a devolve ao Vite quando não encontra o endpoint.

C)

O proxy altera apenas a URL exibida no navegador, sem encaminhar a requisição ao backend.

D)

O Vite cria cópias dos endpoints ASP.NET Core e executa a lógica da API dentro do processo do frontend.

E)

O navegador solicita /api/... ao servidor de desenvolvimento do Vite, e o proxy encaminha a requisição ao backend configurado.

Exercício 3:

Considere o componente React:

```
const [products, setProducts] = useState([]);  
const [page, setPage] = useState(1);  
const [category, setCategory] = useState("");  
const [loading, setLoading] = useState(false);  
const [error, setError] = useState(null);
```

```
useEffect(() => {  
  setLoading(true);  
  setError(null);
```

```
  listarProdutos({ page, category })  
    .then(setProducts)  
    .catch(setError)  
    .finally(() => setLoading(false));  
}, [page, category]);
```

Desconsidere eventuais execuções adicionais realizadas pelo React em modo de desenvolvimento para detectar problemas nos efeitos.

Depois da execução inicial, quais alterações fazem esse efeito ser executado novamente em razão do array de dependências?

A)

Alterações em products ou loading.

B)

Alterações em error ou products.

C)

Qualquer alteração em um estado declarado pelo componente.

D)

Alterações em page ou category.

E)

Apenas alterações em page, porque category contém uma string.

Exercício 4:

Uma tela de listagem precisa distinguir quatro situações:

1. requisição em andamento;
2. requisição concluída com registros;
3. requisição concluída sem registros;
4. requisição que falhou.

Qual estratégia de estado representa melhor essa necessidade?

A)

Manter estados explícitos para carregamento e erro, tratar os dados em caso de sucesso e exibir "nenhum resultado" somente quando não houver carregamento, erro ou itens.

B)

Manter apenas products e interpretar uma lista vazia conforme o tempo transcorrido desde o início da requisição.

C)

Mostrar a mesma mensagem para os estados 1, 3 e 4 e renderizar os dados somente no estado 2.

D)

Interpretar uma resposta bem-sucedida com coleção vazia como falha HTTP, pois sucesso pressupõe pelo menos um registro.

E)

Definir loading como false antes de iniciar a requisição para evitar um estado intermediário na interface.

Exercício 5:

Um frontend React possui:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={ <ProductListPage /> } />
    <Route
      path="/products/:id"
      element={ <ProductDetailsPage /> }
    />
  </Routes>
</BrowserRouter>
```

A página de detalhes contém:

```
const { id } = useParams();

useEffect(() => {
  const productId = Number(id);

  if (!Number.isInteger(productId) || productId <= 0) {
    setError('Identificador inválido');
    return;
  }

  obterProduto(productId)
    .then(setProduct)
    .catch(setError);
}, [id]);
```

Qual alternativa descreve corretamente a divisão de responsabilidades?

A)

/products/:id é o endpoint da API, portanto o ASP.NET Core valida o parâmetro antes de o componente React ser selecionado.

B)

BrowserRouter substitui o roteamento do ASP.NET Core e passa a atender também às URLs do backend.

C)

O roteador do frontend escolhe o componente conforme a URL; a página obtém o parâmetro e pode fazer uma requisição independente ao endpoint da API.

D)

obterProduto() precisa alterar a URL do navegador para /api/... antes de realizar a requisição.

E)

useParams() converte automaticamente parâmetros numéricos para number, tornando a conversão explícita desnecessária.

Exercício 6:

Um usuário efetuou login corretamente. Sua identidade foi reconhecida e os dados enviados por ele são válidos. Entretanto, esse usuário não possui permissão para alterar determinado recurso.

Qual mecanismo deve impedir a operação?

A)

Autenticação, porque qualquer acesso negado significa que a identidade não foi confirmada.

B)

Validação, porque permissões são tratadas como restrições dos dados enviados.

C)

Sessão, porque somente dados armazenados nela podem estabelecer permissões.

D)

Model binding, porque ele determina quais funcionalidades podem ser executadas pelo usuário.

E)

Autorização, porque a identidade já foi confirmada e é necessário verificar se ela possui acesso à operação.

Exercício 7:

Considere dois cenários:

Aplicação A: páginas e formulários utilizados diretamente em um navegador.

Aplicação B: API consumida por um aplicativo móvel e por outros clientes independentes.

Qual alternativa descreve corretamente um uso comum de cookies de autenticação e tokens JWT?

A)

O navegador envia automaticamente qualquer JWT armazenado pelo cliente, enquanto cookies precisam ser inseridos manualmente no cabeçalho Authorization.

B)

Cookies de autenticação são utilizados somente para sessões anônimas, enquanto JWT é necessário para qualquer usuário identificado.

C)

Em aplicações web, o navegador pode enviar automaticamente o cookie de autenticação quando suas regras permitem. Em APIs que usam JWT como bearer token, o cliente normalmente envia o token no cabeçalho Authorization.

D)

Cookies podem armazenar somente roles, enquanto tokens JWT podem transportar apenas claims que não identificam o usuário.

E)

Depois que um cookie ou JWT é emitido, o servidor não precisa validar a credencial nas requisições posteriores.

Exercício 8:

Uma aplicação configurou o pipeline assim:

```
app.UseAuthorization();
```

```
app.UseAuthentication();
```

```
app.MapControllerRoute(
```

```
    name: "default",
```

```
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Uma ação contém:

```
[Authorize(Roles = "Editor,Administrador")]
```

Qual alteração é necessária?

A)

Colocar UseAuthentication() antes de UseAuthorization() para estabelecer a identidade antes da avaliação das regras de acesso.

B)

Remover UseAuthorization(), pois [Authorize] executa toda a autorização sem middleware.

C)

Remover [Authorize], pois autorização baseada em roles é exclusiva de Minimal APIs.

D)

Mapear os controladores antes dos dois middlewares para que a ação seja descoberta antes da autenticação.

E)

Executar UseAuthentication() duas vezes, uma para cada role definida.

Exercício 9:

Uma organização possui a role:

Analista

e a claim:

Permissao = AprovarCadastro

Todos os analistas podem acessar a área de análise, mas somente alguns podem executar a operação específica de aprovação.

Qual desenho é mais adequado?

A)

Utilizar a claim para representar o acesso geral e a role para a operação específica, pois roles são necessariamente mais granulares.

B)

Utilizar somente claims porque a presença de uma claim torna roles incompatíveis com a identidade.

C)

Utilizar somente roles porque claims servem exclusivamente para exibir informações do perfil.

D)

Utilizar a role Analista para o acesso geral e a claim de permissão para representar a capacidade específica de aprovar cadastros.

E)

Enviar role e claim em campos ocultos do formulário para que o próprio usuário informe suas permissões ao servidor.

Exercício 10:

Uma SPA utiliza autenticação baseada em cookie. Para consultar o usuário atual, executa:

```
fetch('/api/auth/me', {  
  credentials: 'include'  
});
```

O servidor protege os recursos que exigem autenticação. No frontend, existe também:

```
function ProtectedRoute() {  
  const { authenticated } = useAuth();
```

```
if (!authenticated)
  return <Navigate to="/entrar" replace />;

return <Outlet />;
}
```

Qual avaliação está correta?

A)

ProtectedRoute impede por si só o acesso aos recursos protegidos, portanto a API não precisa aplicar autorização.

B)

O estado utilizado por ProtectedRoute melhora o fluxo de navegação, mas o servidor continua responsável por proteger os recursos. A opção credentials controla o envio de credenciais nas requisições realizadas por fetch.

C)

credentials: 'include' valida o cookie no navegador antes de a requisição ser enviada, dispensando validação no servidor.

D)

A SPA deve copiar o conteúdo do cookie para localStorage para decidir se uma rota pode ser acessada.

E)

Quando existe uma rota protegida no React, requisições diretas aos endpoints correspondentes também passam pelo ProtectedRoute.